Lecture 03

Object-Oriented Programming

{OOP} Class and Object

Prepared by **Hok Tin**

Content

- Class and Object
  - Creating Class
  - Attributes/Behavior
  - Creating Object
  - Examples
- Static Keyword
- Class Diagrams

Class/Object

❑ What is Class/Object concept in Java?

- A **Class** is a blueprint for the object. Before we create an object, we first need to define the class.

  Example: We can think of the class as a sketch (***prototype***) of a house. It contains all the details about the floors, doors, windows, etc. Based on these descriptions we build the house. House is the ***object***.

House blueprint/prototype

(Class)

(Objects)

Class/Object

❑ When a Class creates an Object

Primitive types (Java built-in)

No need to create

```
class Student {
    int studentID;
    String name;
    char gender;
    int age;
    double score;
    int year;
}
```

Need to create your own

Data Types

- int
- double
- boolean
- String
- Student

Variables

```
int num = 7;
double money = 1.5;
boolean isRaining = true;
String text1 = "Hi";
String text2 = new String("Hello world");

Student stu = new Student();
stu.studentID = 777;
stu.name = "Tola";
stu.gender = 'M';
stu.age = 18;
stu.score = 99.99;
stu.year = 2;
```

Class

Object/Instance

Class/Object

❑ Attributes and Behaviors

- **Attributes** are the individual things that differentiate one object from another and determine the ***appearance***, ***state***, or other ***qualities*** of that object
- **Behavior** determines what instances of that class do when their internal state changes or when that instance ***is asked to do something***

```java
class Car {
    int speed;
    String model;
    boolean isRunning;
    double fuelLevel;

    void drive() {
        speed=1;
        isRunning = true;
    }
    void stop() {
        speed=0;
        isRunning = false;
    }
    void speedUp() {
        speed +=1;
    }
    void fillFuel(double fuel) {
        fuelLevel = fuel;
    }
    double checkFuel() {
        return fuelLevel;
    }
}
```

Attributes/States
(variables of a class)

Behaviors
(Function or method in a class)

Usage:

```java
Car camary = new Car();
camary.speed = 0;
camary.model = "2012 Toyota Camry";
camary.isRunning = false;
camary.fuelLevel = 0;

camary.fillFuel(100);
System.out.println("See fuel: " + camary.checkFuel());

camary.drive();
camary.speedUp();
camary.stop();
```

Class/Object

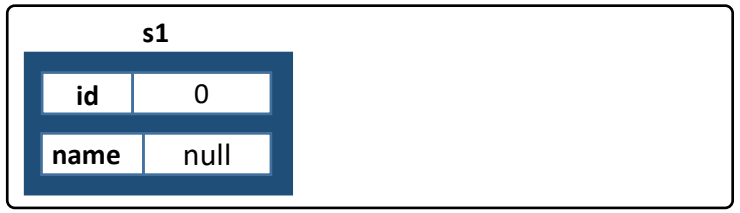
❑ Example: Create a Class and Object

```java
class Student {
    int id;
    String name;
}

public class Main {
    public static void main(String args[]) {

        Student s1 = new Student();

        System.out.println(s1.id);
        System.out.println(s1.name);
    }
}
```

Output:

```
0
null
```

Stack Memory

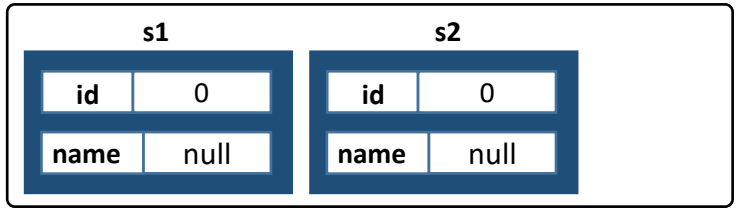



```java
class Student {
    int id;
    String name;
}

public class Main {
    public static void main(String args[]) {

        // Create an object named as s1
        Student s1 = new Student();

        System.out.println(s1.id);
        System.out.println(s1.name);


        // Create another object named as s2
        Student s2 = new Student();

        System.out.println(s2.id);
        System.out.println(s2.name);
    }
}
```

Output:

```
0
null

0
null
```

Stack Memory



Class/Object

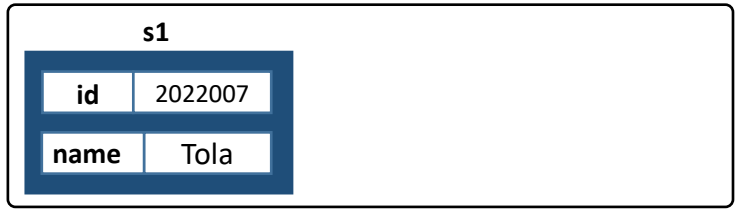
❑ Example: Assigning directly values to objects

```java
class Student {
    int id;
    String name;
}

public class Main {
    public static void main(String args[]) {

        Student s1 = new Student();

        s1.id = 2012007;
        s1.name = "Tola";

        System.out.println(s1.id + " " + s1.name);
    }
}
```

Output:

```
2022007
Tola
```

Stack Memory




```java
class Student {
    int id;
    String name;
}

public class Main {
    public static void main(String args[]) {

        Student s1 = new Student();
        Student s2 = new Student();

        s1.id = 2012007;
        s1.name = "Tola";

        s2.id = 2012001;
        s2.name = "Makara";

        System.out.println(s1.id + " " + s1.name);
        System.out.println(s2.id + " " + s2.name);
    }
}
```

Output:

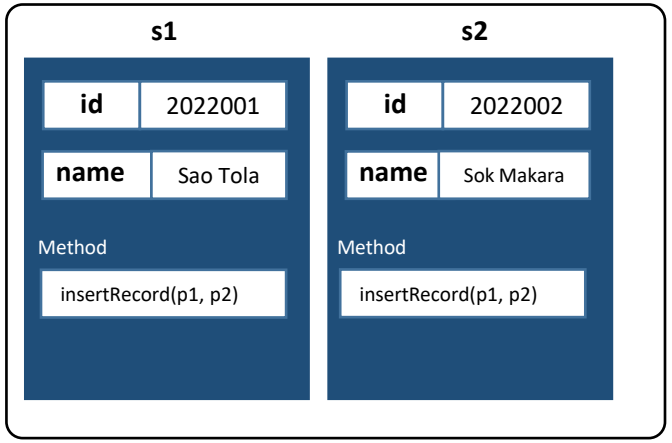
?

Class/Object

❑ Example: Assigning values to an Object through a Method

```java
class Student {
    int id;
    String name;

    void insertRecord(int p1, String p2) {
        id = p1;
        name = p2;
    }
}

public class Test {
    public static void main(String args[]) {

        Student s1 = new Student();
        Student s2 = new Student();

        s1.insertRecord(2022001, "Sao Tola");
        s2.insertRecord(2022002, "Sok Makara");

        System.out.println(s1.id + " " + s1.name);
        System.out.println(s2.id + " " + s2.name);
    }
}
```

Stack Memory

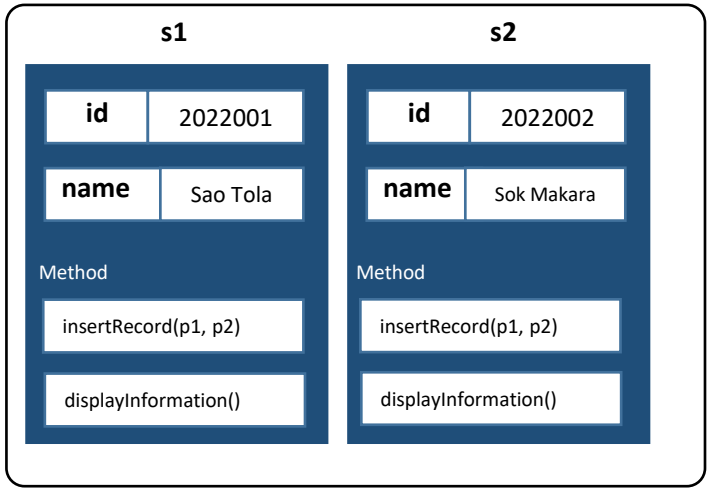


Class/Object

❑ Example: Display values of an Object through a Method

```java
class Student {
    int id;
    String name;

    void insertRecord(int p1, String p2) {
        id = p1;
        name = p2;
    }

    void displayInformation() {
        System.out.println(id + " " + name);
    }
}

public class Main {
    public static void main(String args[]) {

        Student s1 = new Student();
        Student s2 = new Student();

        s1.insertRecord(2022001, "Sao Tola");
        s2.insertRecord(2022002, "Sok Makara");

        s1.displayInformation();
        s2.displayInformation();
    }
}
```

Stack Memory



Static Keyword

❑ Static keyword

- It is independent from any object
- It is used for memory management mainly
- The static keyword belongs to the class than an instance of the class
- It can be used along with ***static variables***, ***static methods***, and ***static blocks***

☞ In Java, if we want to access class members, we must first create an instance of the class, but not for **Static**

```java
class Student {
    int id;
    String name;
    double score;

    static String school;
    static double totalScore;
}
```

Usage:

```java
Student stu1 = new Student();

stu1.id = 111;
stu1.name = "Tola";
stu1.score = 99.99;

Student.school = "Angkor Univ.";
Student.totalScore = 100;
```

Static Keyword

❑ Static variables

If we declare a variable static, all objects of the class ***share the same static variable***

```java
class Student {
    static String name;
    static int age;
}
```

Usage:

```java
Student stu1 = new Student();
Student stu2 = new Student();

stu1.name = "Tola";
stu1.age = 20;

System.out.println("Obj1: " + stu1.name + ","+ stu1.age);

stu2.name = "Makara";
stu2.age = 17;

System.out.println("Obj1: " + stu1.name + ","+ stu1.age);

Student.name = "Kompheak";
Student.age = 28;
System.out.println();
System.out.println("Obj1: " + stu1.name + ","+ stu1.age);
System.out.println("Obj2: " + stu2.name + ","+ stu2.age);
System.out.println("Class: " + Student.name + ","+ Student.age);
```

Output:

```
Obj1: Tola,20
Obj1: Makara,17

Obj1: Kompheak,28
Obj2: Kompheak,28
Class: Kompheak,28
```

Static Keyword

Static methods

A static method can be invoked ***without the need for creating an instance of a class***

```java
class Student {
    String name;
    int age;
    static String school = "Bayon Univ.";

    static void changeSchool(String schoolName) {
        school = schoolName;
    }
}
```

Usage:

```java
Student stu = new Student();
stu.changeSchool("Ta Prohm Univ.");;

Student.changeSchool("Angkor Univ.");
```

Static blocks

The static block ***is executed only once when the class is loaded*** in memory

```java
class Student {
    String name;
    int age;
    static String school = "Bayon Univ.";

    static {
        school = "Angkor Univ.";
    }
}
```

```java
System.out.print(Student.school);
```

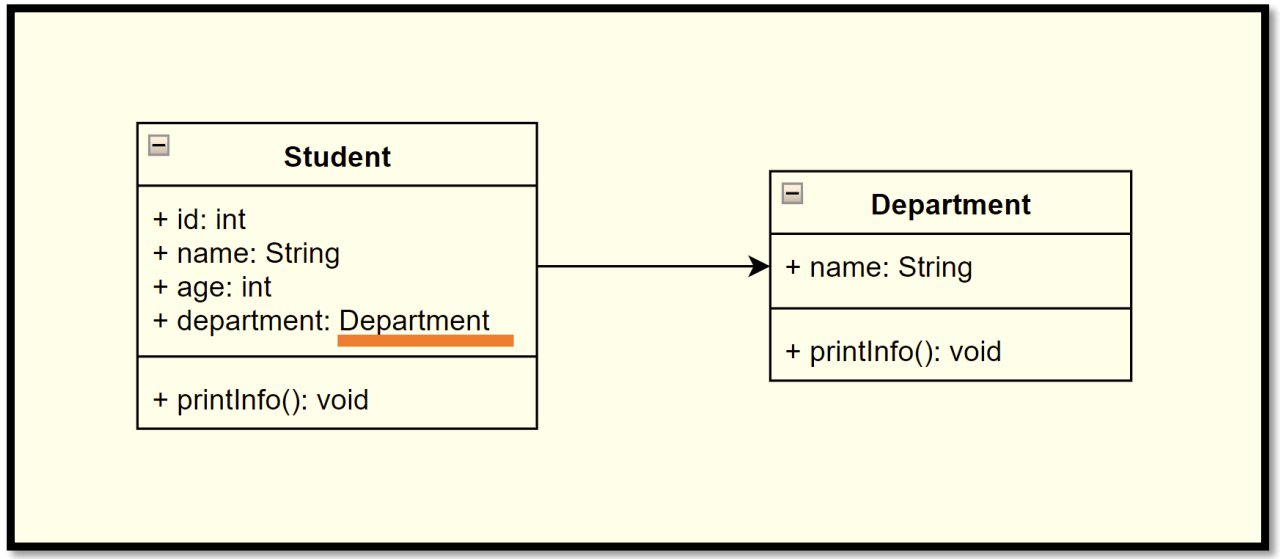
Output:

```
Angkor Univ.
```

Class Diagrams

Class Diagrams

- Enable us to model software in a ***high level of abstraction*** and *without having to look at the source code*
- Designing and modeling software to ***describe classes and their relationships***
- Shows the ***names*** and ***attributes*** of the classes, ***connections*** between the classes, and sometimes also the ***methods*** of the classes



The figure above shows that a *Student knows its Department*, but a *Department does not know about Students* they are the author of

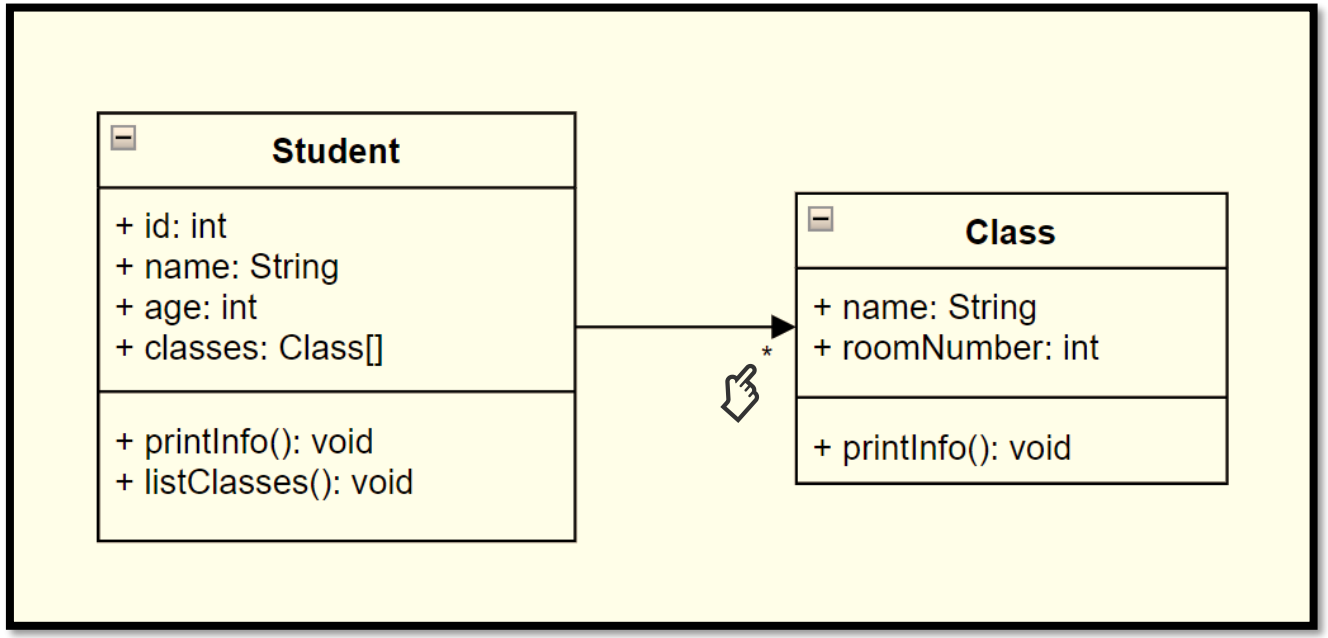
In Java:

```java
class Department {
    String name;
    void printInfo() {
        System.out.println(name);
    }
}

class Student {
    int id;
    String name;
    int age;
    Department department;
    void printInfo() {
        System.out.println(id + " " + name +" "+ age);
        System.out.println(department.name);
    }
}
```

Class Diagrams

Connections between classes

- In a class diagram, the connections between classes are shown as arrows
- The arrows also show the direction of the connection



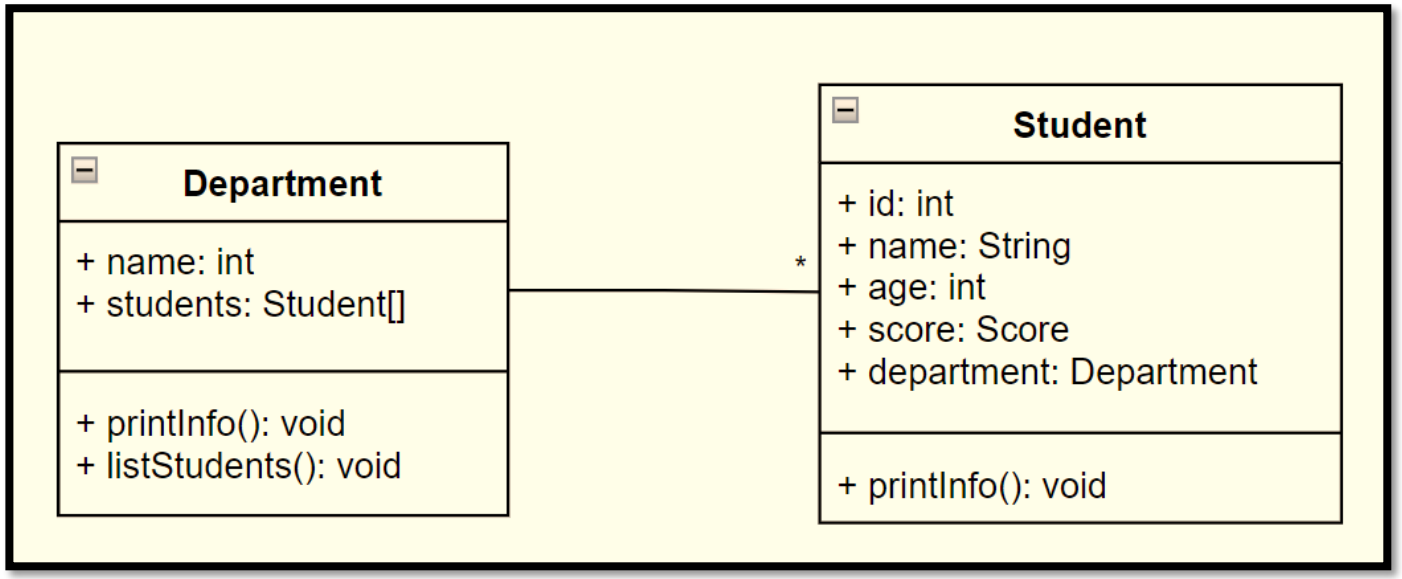
The **star (*)** tells us that a student can have *between 0 and unlimited number* of classes

In Java:

```java
class Class {
    String name;
    int roomNumber;
    void printInfo() {
        System.out.println(name + " " + roomNumber);
    }
}

class Student {
    int id;
    String name;
    int age;
    Class[] classes;
    void printInfo() {
        System.out.println(id + " " + name +" "+ age);
    }
    void listClasses() {
        // loop to display info. of all classes here
    }
}
```

Class Diagrams

❑ Connections between classes



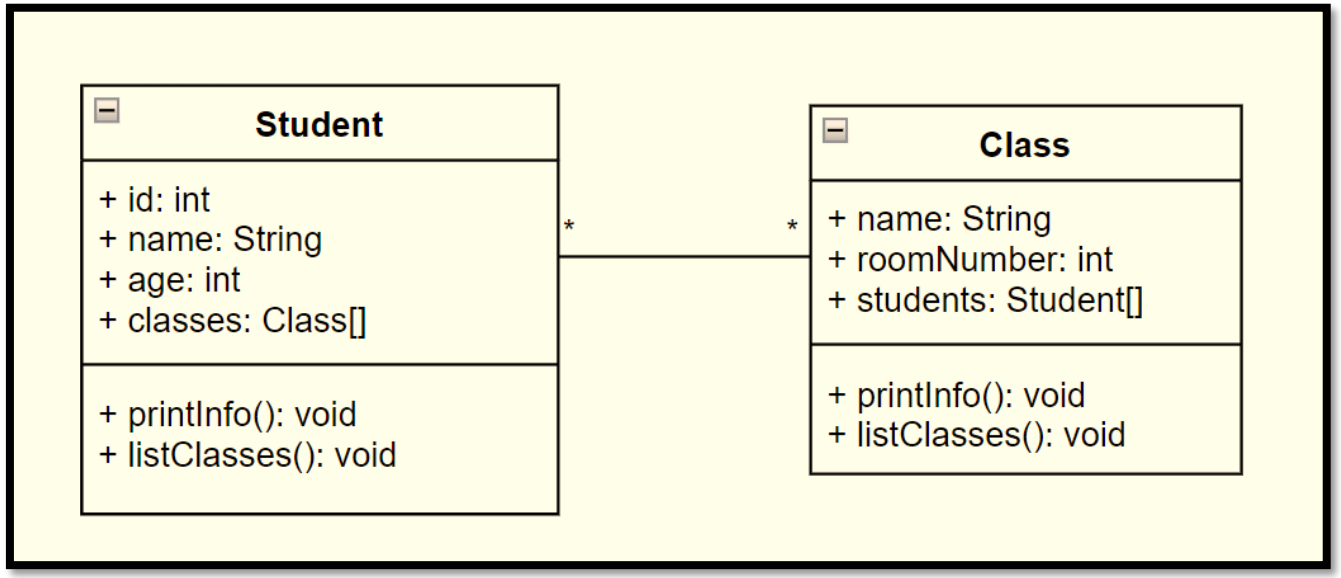
- If there is *no arrowhead* in a connection, both classes *know about each other*.
- Department has multiple students
- Student has only one department

In Java:

```java
class Department {
    String name;
    Student[] students;
    void printInfo() {
        System.out.println(name);
    }
    void listStudents() {
        // Loop to list info. of all students
    }
}

class Student {
    int id;
    String name;
    int age;
    Department department;
    void printInfo() {
        System.out.println(id + " " + name +" "+ age);
        System.out.println(department.name);
    }
}
```

Class Diagrams

❑ Connections between classes



- we *add a star to both ends* of the connection
- A student can have multiple classes
- A student can have multiple students

In Java:

```java
class Class {
    String name;
    int roomNumber;
    Student[] students;
    void printInfo() {
        System.out.println(name + " " + roomNumber);
    }
    void listStudents() {
        // loop to list info. of all students
    }
}

class Student {
    int id;
    String name;
    int age;
    Class[] classes;
    void printInfo() {
        System.out.println(id + " " + name +" "+ age);
    }
    void listClasses() {
        // loop to list info. of all classes
    }
}
```

Class/Object

❑ Examples

```java
class BankAccount {
    String accountHolder;
    double balance;
    String accountType;

    void displayInfo() {
        System.out.println("Account Info: Holder: " + accountHolder);
        System.out.println("Type: " + accountType);
        System.out.println("Balance: " + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount myAccount = new BankAccount();
        myAccount.accountHolder = "John Doe";
        myAccount.accountType = "Savings";
        myAccount.balance = 5000.0;

        myAccount.displayInfo();
    }
}
```

Class/Object

❑ Examples

```java
class Car {
    String make = "Unknown";
    String model = "Unknown";
    int year = 0;


    void insertInfo(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    void displayInfo() {
        System.out.println("Car Info: " + make + " " + model + " " + year);
    }
}

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car();
        car1.displayInfo();

        Car car2 = new Car("Toyota", "Camry", 2022);
        car2.displayInfo();
    }
}
```

Class/Object

❑ Examples

```java
class Person {
    String name;
    int age;

    void setInfo(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Person person1 = new Person();
        person1.setInfo("Alice", 30);
        Person person2 = new Person();
        person2.setInfo("Bob", 25);
        Person person3 = new Person();
        person3.setInfo("Charlie", 35);
        Person person4 = new Person();
        person4.setInfo("David", 40);
        Person person5 = new Person();
        person5.setInfo("Eve", 28);

        Person[] people = {
            person1,
            person2,
            person3,
            person4,
            person5
        };

        for (int i = people.length - 1; i >= 0; i--) {
            people[i].displayInfo();
        }
    }
}
```

Static Keyword

❑ Examples

```java
class MathUtils {
    static int add(int a, int b) {
        return a + b;
    }

    int multiply(int a, int b) {
        return a * b;
    }
}

public class MathTest {
    public static void main(String[] args) {

        int sum = MathUtils.add(5, 3);
        System.out.println("Sum: " + sum);

        MathUtils math = new MathUtils();
        int product = math.multiply(4, 2);
        System.out.println("Product: " + product);

    }
}
```

Static Keyword

- **Examples**

```java
class BankAccount {
    private static double interestRate = 0.05;
    private double balance;

    public void setBalance(double initialBalance) {
        balance = initialBalance;
    }

    static void setInterestRate(double rate) {
        interestRate = rate;
    }

    void applyInterest() {
        balance += balance * interestRate;
    }

    double getBalance() {
        return balance;
    }
}

public class BankAccountTest {
    public static void main(String[] args) {
        BankAccount account1 = new BankAccount();
        account1.setBalance(1000.0);
        BankAccount account2 = new BankAccount();
        account1.setBalance(2000.0);

        System.out.println("Initial Balance (Account 1): " + account1.getBalance());
        account1.applyInterest();
        System.out.println("Balance after Interest (Account 1): " + account1.getBalance());

        System.out.println("Initial Balance (Account 2): " + account2.getBalance());
        BankAccount.setInterestRate(0.07); // Change interest rate
        account2.applyInterest();
        System.out.println("Balance after Interest (Account 2): " + account2.getBalance());
    }
}
```

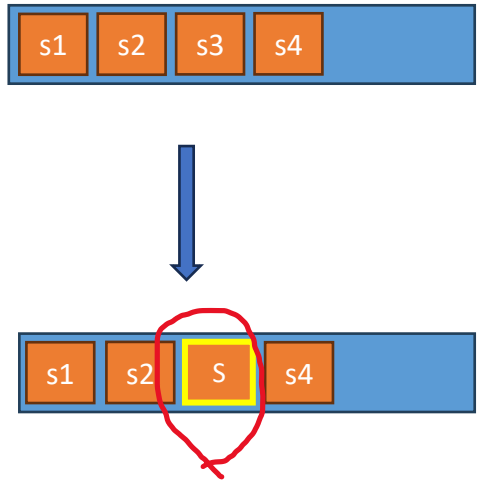
In-class Practice

❑ Replacing in array

```java
class Student {
    private String name;
    private int age;

    public void setStudentInfo(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getStudentInfo() {
        return "Name: " + name + ", Age: " + age;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        int studentTotal = 0;
        Student[] students = new Student[99];

        Student student1 = new Student();
        student1.setStudentInfo("John", 20);
        Student student2 = new Student();
        student2.setStudentInfo("Alice", 22);
        Student student3 = new Student();
        student3.setStudentInfo("Bob", 21);
        Student student4 = new Student();
        student4.setStudentInfo("Eva", 19);

        students[0] = student1;
        studentTotal++;
        students[1] = student2;
        studentTotal++;
        students[2] = student3;
        studentTotal++;
        students[3] = student4;
        studentTotal++;

        // Create a new Student
        Student newStudent = new Student();
        newStudent.setStudentInfo("Tola", 18);

        // Display
        for (int i = 0; i < studentTotal; i++) {
            System.out.println(student.getStudentInfo());
        }
    }
}
```

To replace a student named “Bob” by “newStudent” object in the array



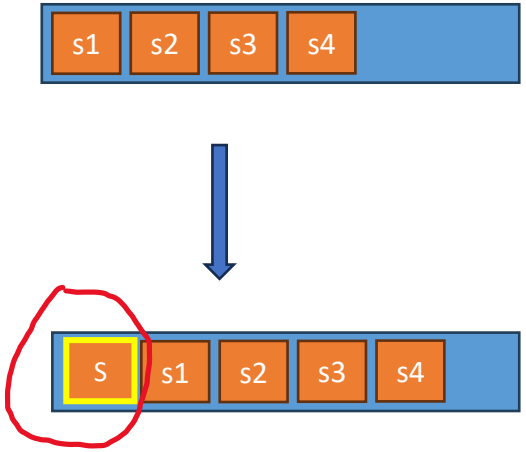
In-class Practice

❑ Shift an object into an array

```java
class Student {
    private String name;
    private int age;

    public void setStudentInfo(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getStudentInfo() {
        return "Name: " + name + ", Age: " + age;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        int studentTotal = 0;
        Student[] students = new Student[99];

        Student student1 = new Student();
        student1.setStudentInfo("John", 20);
        Student student2 = new Student();
        student2.setStudentInfo("Alice", 22);
        Student student3 = new Student();
        student3.setStudentInfo("Bob", 21);
        Student student4 = new Student();
        student4.setStudentInfo("Eva", 19);

        students[0] = student1;
        studentTotal++;
        students[1] = student2;
        studentTotal++;
        students[2] = student3;
        studentTotal++;
        students[3] = student4;
        studentTotal++;

        // Create a new Student
        Student newStudent = new Student();
        newStudent.setStudentInfo("Tola", 18);

        // Display
        for (int i = 0; i < studentTotal; i++) {
            System.out.println(student.getStudentInfo());
        }
    }
}
```

To add “newStudent” to the front of the array



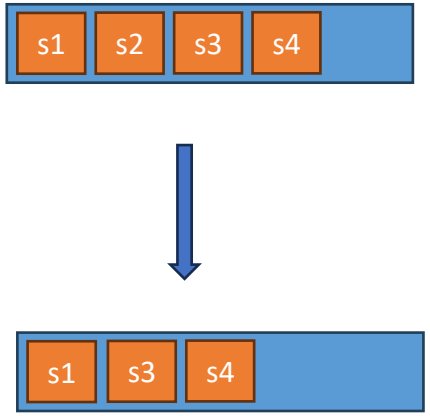
In-class Practice

❑ Delete an object in an array

```java
class Student {
    private String name;
    private int age;

    public void setStudentInfo(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getStudentInfo() {
        return "Name: " + name + ", Age: " + age;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        int studentTotal = 0;
        Student[] students = new Student[99];

        Student student1 = new Student();
        student1.setStudentInfo("John", 20);
        Student student2 = new Student();
        student2.setStudentInfo("Alice", 22);
        Student student3 = new Student();
        student3.setStudentInfo("Bob", 21);
        Student student4 = new Student();
        student4.setStudentInfo("Eva", 19);

        students[0] = student1;
        studentTotal++;
        students[1] = student2;
        studentTotal++;
        students[2] = student3;
        studentTotal++;
        students[3] = student4;
        studentTotal++;

        // Display
        for (int i = 0; i < studentTotal; i++) {
            System.out.println(student.getStudentInfo());
        }
    }
}
```

Delete a student with the name “Alice" in the array



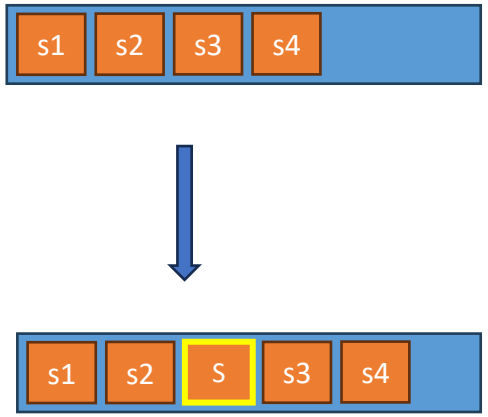
In-class Practice

- Insert an object in an array by index

```java
class Student {
    private String name;
    private int age;

    public void setStudentInfo(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getStudentInfo() {
        return "Name: " + name + ", Age: " + age;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        int studentTotal = 0;
        Student[] students = new Student[99];

        Student student1 = new Student();
        student1.setStudentInfo("John", 20);
        Student student2 = new Student();
        student2.setStudentInfo("Alice", 22);
        Student student3 = new Student();
        student3.setStudentInfo("Bob", 21);
        Student student4 = new Student();
        student4.setStudentInfo("Eva", 19);

        students[0] = student1;
        studentTotal++;
        students[1] = student2;
        studentTotal++;
        students[2] = student3;
        studentTotal++;
        students[3] = student4;
        studentTotal++;

        // Create a new Student
        Student newStudent = new Student();
        newStudent.setStudentInfo("Tola", 18);

        // Display
        for (int i = 0; i < studentTotal; i++) {
            System.out.println(student.getStudentInfo());
        }
    }
}
```

Insert “newStudent” to index 2 of the array



Good luck 👌

😉 **References:**

https://java-programming.mooc.fi/

https://www.javatpoint.com/

https://www.programiz.com/java-programming